

**JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY,
NOIDA**

**DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND
INFORMATION TECHNOLOGY**



Project Title: COMPETITIVE PROGRAMMING ANALYZER LAB

Enrol. No.	Name of Student
2401030071	Ivy Singh
2401030072	Rashi Johari
2401030082	Priyanshu Gumber
2401030376	Ankita Parashar

Course Name: Competitive Programming

Course Code: 24B15CS224

Program: B. Tech. CSE

2nd Year 4th Semester

2025-2026

TABLE OF CONTENTS

S. NO.	TOPIC	PAGE NO.
1.	Problem Statement	3
2.	Significance of Problem	3
3.	Implementation Details	4
4.	Output	6
5.	References	8

1.PROBLEM STATEMENT

Competitive programming requires rigorous testing, optimization, and an in-depth understanding of algorithmic complexity. While modern platforms provide pass/fail verdicts and execution times, they often lack granular feedback on *why* a solution behaves the way it does under various constraints. Programmers frequently struggle to diagnose hidden bottlenecks, estimate empirical Big-O complexity accurately, or identify exact edge cases where their logic fails.

The **Competitive Programming Analyzer Lab** is an automated, full-stack web application designed to solve this problem. It allows users to upload C++ solutions, execute them against custom or automatically generated test suites, and receive a comprehensive algorithmic analysis. The system bridges the gap between static code structures and runtime performance by calculating Big-O complexity estimations, detecting recursion/backtracking depths, and visualizing performance through empirical charts.

2.SIGNIFICANCE OF PROBLEM

The ability to analyze algorithm efficiency is a cornerstone of computer science and software engineering. Without automated tooling, developers rely on manual tracing and informal heuristics, which are prone to error—especially in complex data structures or deeply nested recursive calls.

From an analytical and engineering perspective, creating this system presents several significant challenges:

- **Static Complexity Approximation:** Safely parsing and evaluating C++ code without a full compiler front-end requires sophisticated regex-based static analysis to detect function signatures, loop depths, and recursive call graphs.
- **Empirical Validation:** Statically derived complexity (e.g., $O(n \log n)$) must be cross-verified through runtime execution mapping. This involves running the code against escalating input sizes and applying least-squares regression to fit the results to known mathematical models.
- **Safe Sandboxing:** Compiling and executing user-provided C++ code inherently risks infinite loops and memory bombs, requiring robust isolation, strict timeouts, and memory profiling.
- **Dynamic Test Generation:** Identifying the nature of an algorithm (e.g., Sieve of Eratosthenes, Topological Sort) and automatically generating boundary and stress-test cases tailored to that specific algorithm.

Solving this problem demonstrates a complete engineering lifecycle: from raw file parsing and abstract syntax tree (AST) heuristic construction to safe subprocess execution, data regression analysis, and interactive data visualization.

3.IMPLEMENTATION DETAILS

3.1 Technology Stack

The system is architected as a monorepo containing a React-based frontend and a Node.js backend.

- **Frontend:** Built using React 18 and Vite 5. Tailwind CSS 3 is utilized for utility-first styling. Recharts is used for plotting performance visualizations, and Prism.js is integrated for C++ syntax highlighting within the CodeViewer component.
- **Backend:** A Node.js server powered by Express 5. It handles file uploads via Multer and relies on an asynchronous SQLite wrapper for persistent session management.
- **Execution Engine:** The application utilizes the native `child_process.spawn` module to invoke the GNU C++ compiler (`g++` with C++17 and `-O2` optimization flags). Runaway processes are mitigated using the `tree-kill` library.

3.2 Static Code Analysis Engine

The backend features a custom 1,400-line static analysis module (`analyzer.js`) that performs deep structural scanning of the source code without relying on a heavyweight AST parser:

- **Loop & Function Extraction:** Identifies function signatures and counts `for`, `while`, and `do-while` loops, calculating maximum nesting depth via brace-depth tracking. It classifies loop iterations as linear, logarithmic, or polynomial based on variable updates.
- **Recursion Detection:** Builds a call graph mapping functions to their callees to detect self-recursive and mutually recursive patterns. It derives recurrence relations (e.g., $T(n) = 2T(n/2) + O(n)$ leading to $O(n \log n)$).
- **Complexity Signal Profiling:** Pattern-matches against 9 known algorithm profiles (e.g., Bubble Sort, Merge Sort, Kahn's Topological Sort, Binary Lifting). It merges this heuristic with loop-depth metrics to estimate Best, Average, and Worst-case time complexities, as well as space complexity.

3.3 Input Profiling and Dynamic Test Generation

The system intelligently infers the required input structure by scanning `cin >>` streams. It categorizes variables as scalar boundaries, array lengths, or edge counts. Based on the inferred profile and detected algorithm signals, it generates specialized test cases:

- **Tipping-Point Cases:** Boundary constraints (e.g., $n=0$, $n=1$), dense graphs for DFS, or duplicate-heavy arrays for sorting algorithms.
- **Generic Scaling:** Progressively larger datasets (e.g., 5, 100, 10,000) generated automatically to test asymptotic behavior.

3.4 Compilation and Execution Pipeline

The `compiler.js` service standardizes the C++ source by replacing non-portable headers (like `#include`) with standard equivalents. The code is compiled and spawned in an isolated temporary directory. During execution, peak Resident Set Size (RSS) memory is sampled every 60ms. A hard 2-second timeout enforced by a SIGKILL watcher prevents server hangs from infinite loops.

3.5 Empirical Complexity Fitting

When operating in "Generated Suite" mode, the analyzer maps execution times against escalating input sizes. It applies least-squares regression across six mathematical models ($O(1)$, $O(\log N)$, $O(N)$, $O(N \log N)$, $O(N^2)$, $O(N^3)$). The model yielding the highest R^2 correlation coefficient is selected as the empirical time complexity, cross-referencing the static AST prediction.

4. OUTPUT

The lab's web interface delivers a highly detailed, interactive dashboard that visualizes the results of both static analysis and dynamic execution. Below are snapshots of the deployed system demonstrating its core functionalities.

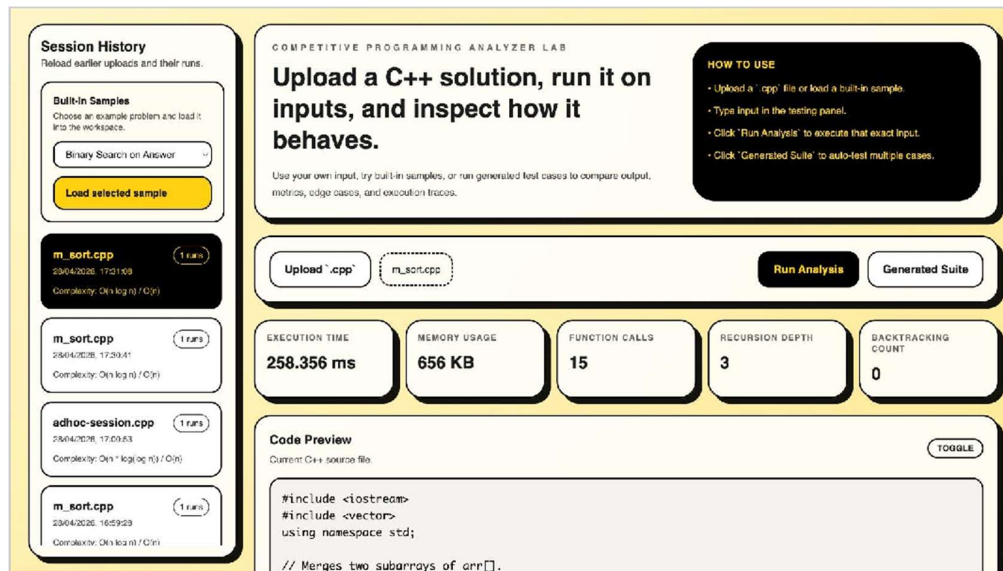


Figure 1: Main Dashboard displaying execution time, memory usage, and function call metrics.

As seen in Figure 1, the primary dashboard provides immediate feedback on physical execution metrics. The system tracks execution time down to milliseconds, peak memory usage in KB, and heuristically measured function calls and recursion depths.

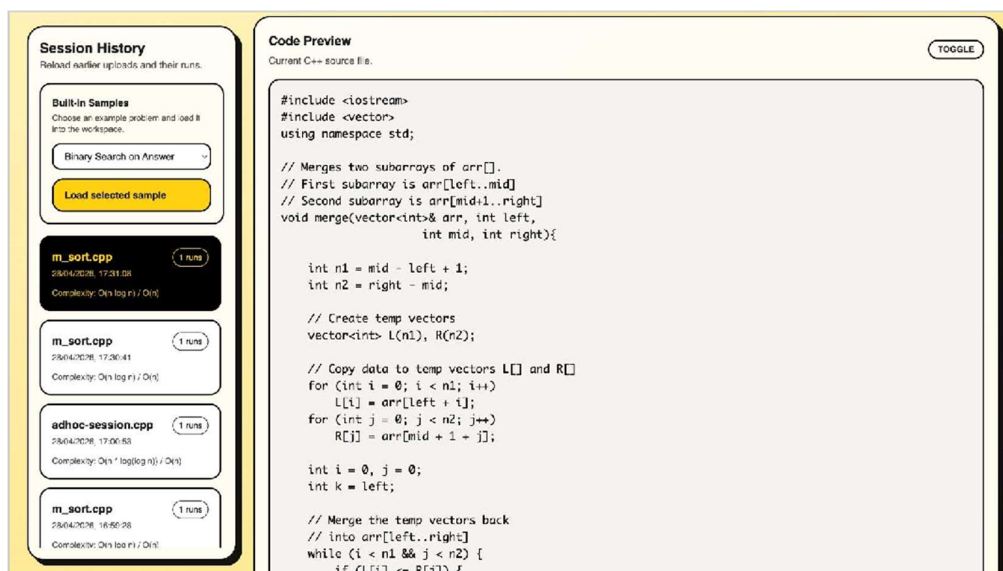


Figure 2: The Code Preview panel utilizing Prism.js for syntax highlighting of the uploaded C++ solution.

Figure 2 illustrates the integrated Code Viewer. The source code (in this case, an implementation of Merge Sort) is presented with accurate C++ syntax highlighting.

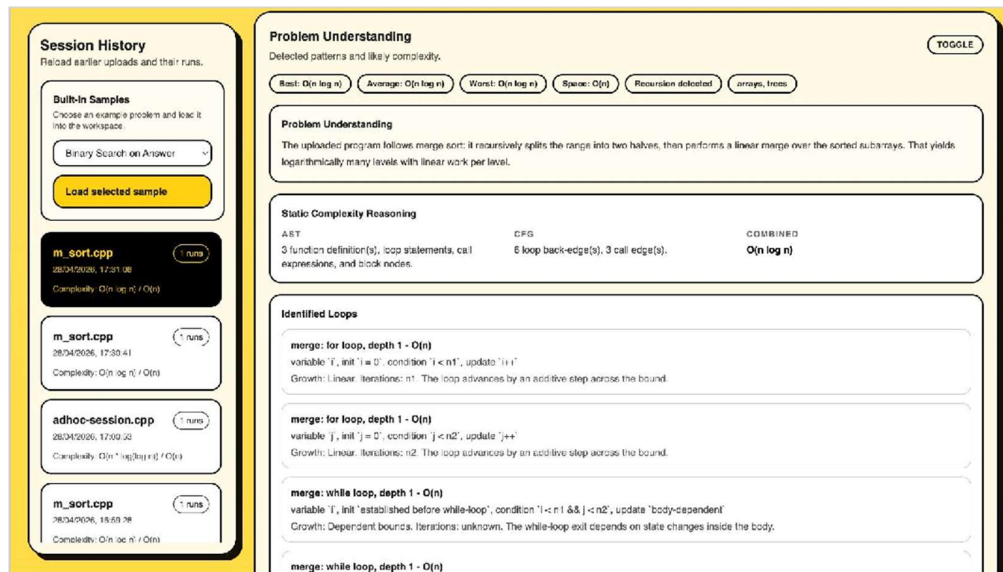


Figure 3: Static Complexity Reasoning and Problem Understanding module.

Figure 3 highlights the core analytical output. The static analysis engine has correctly identified the algorithm as following a split-and-merge paradigm, deducing the Best, Average, and Worst-case time complexities as $O(n \log n)$ and Space complexity as $O(n)$.

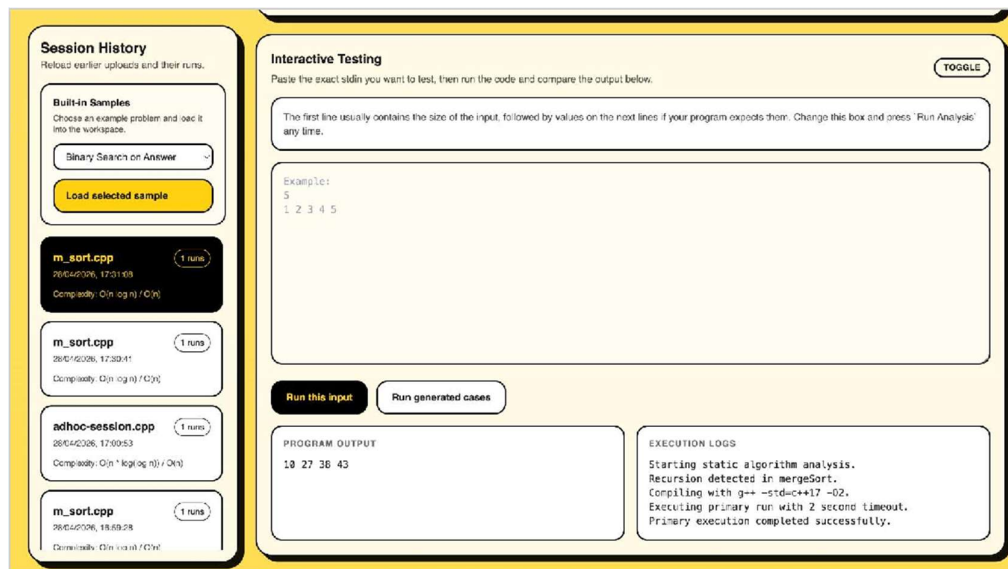


Figure 4: Interactive Testing interface allowing custom stdin execution.

Finally, Figure 4 demonstrates the Interactive Testing panel where users can provide custom standard input ('stdin'). The system executes the compiled binary, returning the exact standard output ('stdout') alongside a detailed execution log.

5. REFERENCES

1. React Documentation. (2025). *React: The library for web and native user interfaces*. Retrieved from <https://react.dev/>
2. Free Software Foundation. (2025). *GCC, the GNU Compiler Collection*. Retrieved from <https://gcc.gnu.org/>
3. Vite Tooling. (2025). *Vite: Next Generation Frontend Tooling*. Retrieved from <https://vitejs.dev/>
4. Express API Reference. (2025). *Express - Node.js web application framework*. Retrieved from <https://expressjs.com/>
5. Recharts. (2025). *A composable charting library built on React components*. Retrieved from <https://recharts.org/>
6. Prism.js. (2025). *Lightweight, robust, elegant syntax highlighting*. Retrieved from <https://prismjs.com/>
7. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.